08.1 — Test Plan

> **For:** Anyone who changes Haritha Hospitals ERPNext — admins, devs, the operator running deploys. **Goal:** Verify changes work correctly, don't break existing functionality, and meet business needs. **Last updated:** 2026-08-29 **Pre-req reading:** 02.1 Shift Management Workflow, 04.1 Deployment Runbook, LEARNINGS #48 (8-phase test playbook)

Purpose

This test plan defines **how** and **when** Haritha Hospitals ERPNext is tested after every change. It exists to:

- Catch regressions before they reach users
- Verify new functionality works as designed
- Provide auditable evidence (test results) that the system is healthy
- Guide the operator on which tests to run for which change type

Without testing, deploys are guesses. With testing, deploys are confidence.

Test strategy

Three principles guide every test decision:

1. Risk-based testing

Allocate testing effort proportional to risk:

Risk Level	Example Changes	Test Effort
Critical	Schema migration, payroll calc, security fix	Full 8-phase (4 hours)
High	New module install, custom field on Employee/Payroll, infra change	8-phase lite (1.5 hours)
Medium	UI text change, new print format, holiday list add	Smoke + relevant functional (30 min)
Low	CSS tweak, doc update, env var rename	Smoke only (5 min)

2. Shift-left (test early, not late)

Test in **dev first**, not after deploy:

- Devs write unit/integration tests as part of code change
- 8-phase test runs on **pberpdev** before promote to QA
- Smoke test on **pberppprod** only catches deploy-time surprises

  Late testing = surprise incidents. Early testing = planned fixes.

3. Continuous (every change triggers relevant tests)

Every code change → relevant tests run automatically:

- PR merged → CI runs unit + smoke
- Deploy to pberpqa → 8-phase lite
- Deploy to pberpprod → smoke + monitoring

No "I'll test it later" — tests run as part of the change flow.

Test levels

Four levels, applied by change type:

Unit tests

What: Individual Python functions, JS components. **Where:** `pytest` (Python), `jest` (JS). **Who:** Developer. **When:** Before PR merge. **Coverage target:** 60% (baseline; increase per module as bugs emerge).

```
cd /home/vijay/frappeclaw/apps/haritha_hospital
pytest -v  # Python tests
cd frontend && npm test  # JS tests
```

Integration tests

What: Frappe API endpoints, scheduled jobs, cross-DocType flows. **Where:** `bench --site X execute` or `bench --site X run-tests`. **Who:** Developer + admin. **When:** After unit tests pass, before deploy.

```
bench --site pberpDEV.duckdns.org execute haritha_hospital.tests.test_integration.run_all
```

System tests (end-to-end via browser)

What: Real user flows in real UI (click through, verify behavior). **Where:** Manual or Playwright. **Who:** Admin (Venkat). **When:** Pre-deploy to prod, after deploy to prod. **Tool:** Browser + screen recorder for evidence.

UAT (User Acceptance Testing)

What: Business user validates that the change meets their need. **Where:** pberpqa or staged-prod. **Who:** Venkat (designated business user). **When:** Before prod deploy for major changes.

Test environment requirements

Env	Use	Notes
pberpdev	Post-deploy validation, fixture testing	Schema experiments OK
pberpqa	Pre-prod gate	Currently skipped for Haritha work — light use

Env	Use	Notes
pberpprod	Post-deploy smoke only	**No destructive testing here**

> **Critical rule:** Never run a test that mutates data on pberpprod. Read-only smoke only.

Entry criteria

Testing starts when:

- ☐ Code merged to `main` (or relevant branch)
- ☐ Deployable artifacts ready (Docker image, fixtures, code changes)
- ☐ Environment prepared (target env stable, no in-flight jobs)
- ☐ Test plan selected (smoke / 8-phase / specific cases)
- ☐ Tester identified (admin for system tests, Venkat for UAT)

If any criterion is missing, defer testing.

Exit criteria

Testing ends when **all** of:

- ☐ All selected tests executed (no skipped without reason)
- ☐ All SEV-1 (critical) tests pass
- ☐ All SEV-2 (high) tests pass — or known failures with documented workaround
- ☐ No SEV-1 or SEV-2 defects open
- ☐ Test results documented (in PR comment, runbook log, or `docs/TEST_RESULTS.md`)
- ☐ Sign-off from Venkat (for major changes)

A test cycle is "passed" only when exit criteria met. Otherwise it's "blocked" — fix and re-test.

Risk-based testing matrix

Change Type	Test Plan	Time	Owner
Schema migration (e.g. new version of Frappe/ERPNext/HRMS)	8-phase full	4 hours	Admin
New module install (e.g. bench `install-app`)	8-phase (LEARNINGS #48)	1 day	Admin
Custom field add (e.g. new CF on Employee)	Phase 1 + Phase 3 + Phase 4 (Employee CRUD)	30 min	Admin
Property setter (e.g. shift color)	Phase 1 + manual verify roster SPA	15 min	Admin
Print format	Phase 1 + sample PDF render	15 min	Admin

Change Type	Test Plan	Time	Owner
Salary structure change	Phase 1 + payroll smoke + 1 sample payslip	30 min	Payroll + Admin
Holiday list update	Phase 1 + list view verify	5 min	Admin
UI text change	Phase 1 only	5 min	Admin
DocType rename	8-phase full + data integrity check	4 hours	Admin
Backup / restore	Phase 1 + restore to scratch env	2 hours	Admin
DB migration (large data move)	8-phase + manual data sample	4 hours	Admin
Quarterly regression	8-phase full + Phase 7 (60+ test cases)	1 day	Venkat

---

Test schedule

Cadence	Test	Time	Notes
Per PR	Smoke (`pytest` + curl)	5 min	CI runs
Per deploy	8-phase (Phases 1–6)	4 hours	Admin
Monthly	Phase 7 regression (60+ test cases from 08.2)	4 hours	Admin
Quarterly	Full 8-phase + UAT	1 day	Venkat
Annually	DR drill (restore from backup to scratch env)	2 days	Venkat

> **Smoke is mandatory per deploy.** Everything else scales by risk.

Roles

Developer

- Writes unit tests (Python + JS)
- Writes integration tests for new features
- Fixes bugs found in any test level
- Updates test docs when adding new functionality

Admin (Venkat + ERPClaw subagents)

- Runs system tests (8-phase, manual, Playwright)
- Runs vendor tests (`bench run-tests --app <X>`)
- Triages defects, assigns severity
- Maintains test scripts and fixtures
- Logs test results

Venkat (UAT owner)

- Approves UAT for major changes
- Final sign-off before prod deploy
- Reviews quarterly regression results
- Owns test process improvements

Defect management

Severity levels

Severity	Definition	Example	Response
SEV-1 (Critical)	System unusable / data loss	Login broken, payroll miscalc, DB corruption	Stop deploy. Fix immediately.
SEV-2 (High)	Major feature broken, no workaround	Roster SPA blank, leave approval fails	Fix before deploy.
SEV-3 (Medium)	Feature broken, workaround exists	One custom field mislabeled	Fix in next sprint.
SEV-4 (Low)	Cosmetic / minor	Typo, alignment issue	Backlog.

Defect log

Every defect gets:

- **Title:** one-line summary
- **Severity:** SEV-1 to SEV-4
- **Steps to reproduce:** numbered list
- **Expected:** what should happen
- **Actual:** what does happen
- **Screenshot/log:** evidence
- **Environment:** pberpprod / pberpqa / pberpdev
- **Reporter:** name
- **Status:** Open / In Progress / Fixed / Verified / Closed

Tracked in: GitHub Issues (preferred) or `docs/DEFECTS.md` (fallback).

Resolution flow

```
Open → Triage (admin) → Assign severity → Fix (dev) → Verify (admin) → Close
                                                            ↑
                                                  Re-test if needed
```

Test reporting

Every test cycle produces a report. Minimum fields:

Field	Description
Test cycle ID	`TEST-YYYYMMDD-<seq>`

Field	Description
Date / Time	ISO timestamps
Tester	Name
Env tested	pberpdev / pberpqa / pberpprod
Change context	PR #, deploy ID, or "Quarterly regression"
Plan executed	Smoke / 8-phase lite / 8-phase full / Custom
Tests run	Count
Passed	Count
Failed	Count (with severity)
Blockers	List (severity + owner)
Sign-off	Venkat name + date (for major changes)

Save report at: `docs/TEST_RESULTS_<CYCLE_ID>.md` or PR comment.

Sample report template:

```
## Test Cycle TEST-20260829-001

**Date:** 2026-08-29 14:00–17:30 IST
**Tester:** Venkat (via ERPClaw subagent)
**Env:** pberpprod (post-deploy smoke)
**Context:** haritha_hospital app install + 274 fixture load
**Plan:** 8-phase lite

### Results

| Phase | Tests | Passed | Failed | Notes |
|---|---|---|---|---|
| 1. Smoke | 6 | 6 | 0 | All 200 OK |
| 2. Vendor | 0 | — | — | Skipped (no `run-tests` for custom app yet) |
| 3. Schema | 5 | 5 | 0 | 78 CF + 189 PS + 3 PF + 2 N + 2 LH verified |
| 4. CRUD | 12 | 12 | 0 | Employee/Shift/Leave/Attendance OK |
| 5. Workflow | 6 | 6 | 0 | Leave approval, payroll submit |
| 6. Integration | 4 | 4 | 0 | Checkin → attendance OK |
| 7. Regression | — | — | — | Deferred (next monthly) |
| 8. Docs | 2 | 2 | 0 | LEARNINGS #153 + #154 added |

**Total:** 35/35 passed. **Status:**  GREEN — deploy approved.

**Sign-off:** Venkat, 2026-08-29 17:30 IST
```

Continuous improvement

Every test cycle yields lessons:

- New failure mode → add to LEARNINGS.md
- Slow test → optimize or split
- False positive → refine test
- Coverage gap → add test

LEARNINGS.md is the institutional memory. Always grep it before debugging.

Where to next

- **Test cases catalog?** See 08.2 Test Cases
- **8-phase playbook?** See 08.3 Regression Test Script
- **Pre-deploy?** See 04.1 Deployment Runbook
- **Post-incident?** See 04.4 Incident Response Plan